

Міністерство освіти і науки України
Львівський національний університет імені Івана Франка
Факультет електроніки і комп'ютерних технологій

Звіт
про виконання лабораторної роботи №8
з курсу «Хмарні технології та великі дані»
на тему:
«Future (Майбутнє значення / Promise)»

Виконав:
студент групи ФеІ-42
Трофимов Володимир
Перевірив:
ас. Керод Т. І.

Зміст

Тема.....	3
Мета	3
Вхідні дані	3
Завдання.....	4
Критерії виконання роботи.....	4
Теоретичні відомості	4
Структура програмної реалізації	5
Тестування програмної реалізації.....	6
Висновок	9
Код програми	10

Тема

Асинхронне представлення відкладених обчислень та керування життєвим циклом майбутніх значень за допомогою патерну Future/Promise.

Мета

Дослідження фундаментальних засад асинхронного програмування, моделі конкурентності Event Loop та розробка типізованого класу CustomFuture<T> мовою TypeScript з нуля без використання нативних системних промісів для внутрішньої бізнес-логіки. Реалізація детермінованого тристадійного життєвого циклу (Pending, Fulfilled, Rejected), ланцюжків обробників .then() та .catch() із підтримкою розгортання вкладених об'єктів (adoption), механізму очікування з тайм-аутом .get(timeoutMs), паралельних комбінаторів .all() та .race(). Проведення дворівневої верифікації системи: за допомогою набору модульних тестів Jest (npm test) та практичної демонстрації через HTTP REST API (curl та системні логи сервера).

Вхідні дані

Параметр	Значення	Призначення
FutureState	PENDING, FULFILLED, REJECTED	Перелічуваний тип для фіксації поточного стану життєвого циклу
submit(fn)	fn: () => T Promise<T>	Фабричний метод/диспетчер для запуску асинхронної задачі у фоні (setTimeout(..., 0)).
onSuccess	(value: T) => U CustomFuture<U>	Функція трансформації результату при переході у FULFILLED
onError	(err: any) => U CustomFuture<U>	Обробник виняткових ситуацій для перехоплення помилок у ланцюжку
get(timeoutMs?)	Promise<T>	Адаптерний метод очікування результату з контролем максимального часу виконання.
TimeoutError	Custom Error	Спеціалізований клас винятку, що викидається при перевищенні ліміту очікування.
CustomFuture.all	CustomFuture<T>[] -> CustomFuture<T[]>	Комбінатор бар'єрної синхронізації: чекає завершення всіх задач або відхиляється при першій помилці (fail-fast).

CustomFuture.race	CustomFuture<T>[] -> CustomFuture<T>	Комбінатор змагання: переходить у стан задачі, яка фінішувала першою
-------------------	--------------------------------------	--

Завдання

- Розробити типізований базовий клас CustomFuture<T> мовою TypeScript із підтримкою трьох станів і гарантією одноразовості зміни стану (імутабельність фінального результату).
- Реалізувати диспетчер submit(fn), що ініціює асинхронне виконання обчислення у фоновій черзі подій Node.js.
- Забезпечити підтримку ланцюгових викликів .then() та .catch() із коректною пропagaцією значень та помилок і можливістю усиновлення (adopt) вкладених CustomFuture.
- Реалізувати метод адаптації .get(timeoutMs), який перериває очікування викиданням TimeoutError, не перериваючи виконання самої фонові задачі.
- Реалізувати статичні комбінатори CustomFuture.all та CustomFuture.race.
- Написати повний набір модульних тестів (Jest) у файлі tests/CustomFuture.test.ts для верифікації всіх крайових випадків та переходів станів.
- Створити сервер демонстрації src/server.ts для перевірки поведінки сценаріїв під навантаженням через HTTP-запити (curl).

Критерії виконання роботи

- *Одноразовість переходу (Immutability)*: Стан змінюється строго один раз (PENDING → FULFILLED або PENDING → REJECTED); спроба повторно виконати resolve чи reject не змінює первинне значення/помилку.
- *Пропagaція помилок (Error Propagation)*: Будь-який виняток усередині таски або обробника транслюється вниз по ланцюжку та перехоплюється найближчим .catch() або відхиляє проміс методу .get().
- *Fail-Fast поведінка all*: Якщо хоча б один Future у наборі зазнає краху, CustomFuture.all завершується помилкою негайно, не чекаючи завершення повільніших успішних задач.
- *First-Completion у race*: Результатом комбінатора race стає результат тієї задачі, яка завершилася найшвидше.
- *Коректність тайм-ауту*: Виклик .get(timeoutMs) гарантовано викидає TimeoutError, якщо час вичерпано. При цьому сам Future зберігає здатність завершитися пізніше і віддати значення наступному виклику .get().

Теоретичні відомості

Патерн Future / Promise (Майбутнє значення) інкапсулює значення або помилку, які стануть відомими пізніше в результаті асинхронного обчислення. Робота спирається на модель скінченного автомата з трьох станів:

$$\text{PENDING} \rightarrow \begin{cases} \text{FULFILLED} & (\text{успіх з отриманням результату } T) \\ \text{REJECTED} & (\text{аварія з винятком } E) \end{cases}$$

Ключові аспекти внутрішньої архітектури CustomFuture:

- *Черга слухачів (Listener Queue)*: Замість нативного планувальника промісів використовується власний масив підписників `listeners: Listener[]`. Якщо операція ще триває (PENDING), колбек буферизується в масив. Коли результат отримано, метод `notify()` вивантажує колбеки у макротаск-чергу середовища виконання через `setTimeout(..., 0)`.
- *Адаптерне зв'язування (Result Adoption)*: Якщо колбек `.then()` або `.catch()` повертає не примітив, а інший об'єкт CustomFuture, батьківський майбутній результат не закривається негайно, а «усиновлює» дочірній об'єкт через внутрішній метод `adoptResult`, чекаючи його остаточного вирішення.
- *Паралельна синхронізація*:
 - Бар'єрна синхронізація (all): Вимагає підрахунку залишкових задач `remaining`. Час очікування визначається найповільнішою операцією:

$$T_{\text{all}} \approx \max(t_1, t_2, \dots, t_n)$$

- Селективна синхронізація (race): Фіксує результат першої задачі за допомогою прапорця завершення `finished = true`:

$$T_{\text{race}} \approx \min(t_1, t_2, \dots, t_n)$$

Структура програмної реалізації

Проект має модульну структуру з чітким розділенням відповідальності:

- `src/CustomFuture.ts` — ядро патерну:
 - Клас `CustomFuture<T>` із внутрішнім масивом слухачів `listeners`.
 - Методи життєвого циклу `resolve()` та `reject()` з перевіркою імутабельності стану.
 - Методи трансформації `.then()`, відновлення після помилок `.catch()` та очікування `.get()`.
 - Статичні методи `CustomFuture.all` та `CustomFuture.race`.
 - Фабрика асинхронного запуску `submit(fn)`.
- `tests/CustomFuture.test.ts` — повний набір модульних тестів (Jest) із 14 тест-кейсів, що покривають усю функціональність класу.
- `src/server.ts` — демонстраційний HTTP-сервер на базі вбудованого модуля `node:http` з детальним ISO-логуванням подій для перевірки через клієнтські запити.
- `package.json` — маніфест проекту, що описує залежності (ts-jest, typescript, tsx) та скрипти запуску (`npm test`, `npm run dev`).

Тестування програмної реалізації

1. Комплексне модульне тестування (Jest Test Suite)

Модульне тестування виконувалося за допомогою фреймворку Jest за командою `npm test`. Тест-сьют перевіряє 14 сценаріїв, що покривають усі вимоги технічного завдання:

- Успішне обчислення значення та перехід стану в `FULFILLED`.
- Підтримка ланцюгових викликів `.then()` із послідовною трансформацією.
- Перехоплення та відновлення після винятків через `.catch()`.
- Переведення у стан `REJECTED` та викидання помилки в `.get()`.
- Гарантія одноразового переходу стану: спроба викликати `reject` після `resolve` ігнорується.
- Гарантія одноразового переходу стану: спроба викликати `resolve` після `reject` ігнорується.
- `all()` очікує завершення всіх задач і зберігає початковий порядок результатів.
- `all()` аварійно завершується негайно при виникненні першої помилки (`fail-fast`).
- `race()` повертає результат найшвидшої успішної задачі.
- `race()` відхиляється, якщо найшвидша задача впала з помилкою.
- `get(timeoutMs)` перериває очікування і викидає виняток `TimeoutError`.
- Перевищення тайм-ауту в `get(timeoutMs)` не ламає `Future`: наступний виклик `get()` успішно повертає результат після його появи.
- Метод `.then()` коректно розгортає вкладений `CustomFuture` (`adoption`).
- Метод `.catch()` коректно розгортає альтернативний `CustomFuture`.

```
$ npm test

> custom-future-typescript@1.0.0 test
> jest --runInBand

PASS tests/CustomFuture.test.ts
CustomFuture
  ✓ successfully resolves and get() returns the value (9 ms)
  ✓ supports then() chaining (61 ms)
  ✓ propagates errors through catch() (34 ms)
  ✓ get() rejects when the Future is rejected (70 ms)
  ✓ state can transition only once: resolve then reject (5 ms)
  ✓ state can transition only once: reject then resolve (4 ms)
  ✓ all() waits for every Future and preserves input order (200 ms)
  ✓ all() fails fast when one Future rejects (91 ms)
  ✓ race() resolves with the fastest fulfillment (77 ms)
  ✓ race() rejects when the fastest task rejects (76 ms)
  ✓ get(timeoutMs) rejects with TimeoutError (61 ms)
  ✓ get() can still observe a Future after a previous get() timed out (77 ms)
  ✓ then() can adopt another CustomFuture (94 ms)
  ✓ catch() can adopt another CustomFuture (92 ms)

Test Suites: 1 passed, 1 total
Tests: 14 passed, 14 total
Snapshots: 0 total
Time: 3.632 s, estimated 13 s
```

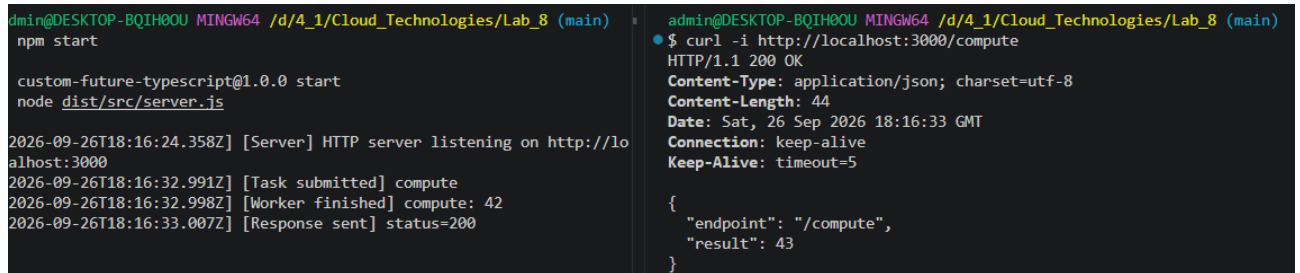
Рис. 1 — Успішне проходження набору модульних тестів Jest

2. Інтеграційне тестування через HTTP-сервер та curl

Для підтвердження коректності роботи механізмів у реальному асинхронному оточенні було запущено тестовий вебсервер (`npm run dev`) та виконано серію контрольних HTTP-запитів.

Дослід 1. Успішне обчислення та ланцюжок `.then()`

Мета: Перевірка обчислення відкладеної операції з подальшою передачею результату по ланцюжку. Обчислюється $21 \times 2 = 42$, після чого в `.then()` додається 1 (очікувано: 43).



```
admin@DESKTOP-BQIH00U MINGW64 /d/4_1/Cloud_Technologies/Lab_8 (main)
npm start

custom-future-typescript@1.0.0 start
node dist/src/server.js

2026-09-26T18:16:24.358Z [Server] HTTP server listening on http://localhost:3000
2026-09-26T18:16:32.991Z [Task submitted] compute
2026-09-26T18:16:32.998Z [Worker finished] compute: 42
2026-09-26T18:16:33.007Z [Response sent] status=200

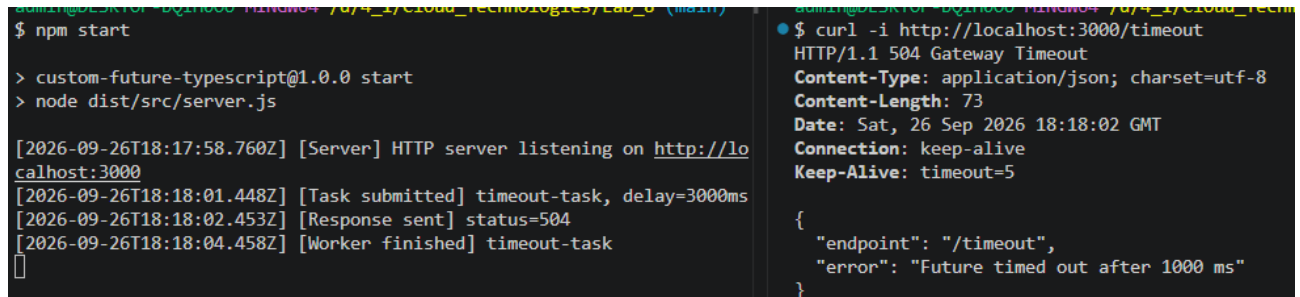
$ curl -i http://localhost:3000/compute
HTTP/1.1 200 OK
Content-Type: application/json; charset=utf-8
Content-Length: 44
Date: Sat, 26 Sep 2026 18:16:33 GMT
Connection: keep-alive
Keep-Alive: timeout=5

{
  "endpoint": "/compute",
  "result": 43
}
```

Рис. 2 — Успішне виконання фонові задачі та ланцюжка `.then()`

Дослід 2. Перевірка механізму тайм-ауту (`.get(timeoutMs)`)

Мета: Перевірка переривання очікування клієнта у випадку, коли тривалість задачі перевищує встановлений ліміт. Запускається задача тривалістю 3000 мс із лімітом очікування `.get(1000)`.



```
admin@DESKTOP-BQIH00U MINGW64 /d/4_1/Cloud_Technologies/Lab_8 (main)
$ npm start

> custom-future-typescript@1.0.0 start
> node dist/src/server.js

[2026-09-26T18:17:58.760Z] [Server] HTTP server listening on http://localhost:3000
[2026-09-26T18:18:01.448Z] [Task submitted] timeout-task, delay=3000ms
[2026-09-26T18:18:02.453Z] [Response sent] status=504
[2026-09-26T18:18:04.458Z] [Worker finished] timeout-task

$ curl -i http://localhost:3000/timeout
HTTP/1.1 504 Gateway Timeout
Content-Type: application/json; charset=utf-8
Content-Length: 73
Date: Sat, 26 Sep 2026 18:18:02 GMT
Connection: keep-alive
Keep-Alive: timeout=5

{
  "endpoint": "/timeout",
  "error": "Future timed out after 1000 ms"
}
```

Рис. 3 — Переривання очікування з винятком `TimeoutError` та поверненням коду 504

Дослід 3. Перевірка комбінатора змагання (`CustomFuture.race`)

Мета: Верифікація відбору першого завершеного значення з набору конкурентних операцій. Конкурують `race-fast` (1000 мс) та `race-slow` (3000 мс).

```
admin@DESKTOP-BQIH00U MINGW64 /d/4_1/Cloud_Technologies/Lab_8 (main)
$ npm start

> custom-future-typescript@1.0.0 start
> node dist/src/server.js

[2026-09-26T18:19:01.844Z] [Server] HTTP server listening on http://localhost:3000
[2026-09-26T18:19:04.324Z] [Task submitted] race-fast, delay=1000ms
[2026-09-26T18:19:04.324Z] [Task submitted] race-slow, delay=3000ms
[2026-09-26T18:19:05.332Z] [Worker finished] race-fast
[2026-09-26T18:19:05.364Z] [Worker finished] race outcome=fast task won, elapsed=1041ms
[2026-09-26T18:19:05.372Z] [Response sent] status=200
[2026-09-26T18:19:07.331Z] [Worker finished] race-slow

admin@DESKTOP-BQIH00U MINGW64 /d/4_1/Cloud_Technologies/Lab_8 (main)
$ curl -i http://localhost:3000/race
HTTP/1.1 200 OK
Content-Type: application/json; charset=utf-8
Content-Length: 75
Date: Sat, 26 Sep 2026 18:19:05 GMT
Connection: keep-alive
Keep-Alive: timeout=5

{
  "endpoint": "/race",
  "winner": "fast task won",
  "elapsedMs": 1041
}
```

Рис. 4 — Фіксація результату найшвидшої задачі за допомогою комбінатора *race*

Дослід 4. Перевірка паралельної синхронізації (CustomFuture.all)

Мета: Верифікація конкурентного паралельного виконання трьох операцій тривалістю 1000 мс, 2000 мс та 1500 мс. Очікується, що загальний час виконання становитиме приблизно 2000 мс (тривалість найдовшої задачі), а не їхню суму 4500 мс.

```
admin@DESKTOP-BQIH00U MINGW64 /d/4_1/Cloud_Technologies/Lab_8 (main)
npm start

custom-future-typescript@1.0.0 start
node dist/src/server.js

2026-09-26T18:20:18.114Z [Server] HTTP server listening on http://localhost:3000
2026-09-26T18:20:19.575Z [Task submitted] all-task-1, delay=1000ms
2026-09-26T18:20:19.575Z [Task submitted] all-task-2, delay=2000ms
2026-09-26T18:20:19.576Z [Task submitted] all-task-3, delay=1500ms
2026-09-26T18:20:20.592Z [Worker finished] all-task-1
2026-09-26T18:20:21.091Z [Worker finished] all-task-3
2026-09-26T18:20:21.589Z [Worker finished] all-task-2
2026-09-26T18:20:21.612Z [Response sent] status=200

admin@DESKTOP-BQIH00U MINGW64 /d/4_1/Cloud_Technologies/Lab_8 (main)
$ curl -w "\nTotal curl time: %{time_total}s\n" http://localhost:3000/all
{
  "endpoint": "/all",
  "results": [
    "A",
    "B",
    "C"
  ],
  "elapsedMs": 2030,
  "note": "Tasks run concurrently; total time is approximately the slowest task."
}
Total curl time: 2.046197s
```

Рис. 5 — Бар'єрна синхронізація паралельних завдань комбінатором *all*

Висновок

У ході виконання лабораторної роботи було досліджено теоретичні основи та практичну реалізацію шаблону проєктування Future / Promise (Майбутнє значення) мовою TypeScript.

Було створено типізований клас CustomFuture<T>, внутрішній стан якого функціонує як скінченний автомат із незворотним переходом станів. Реалізований диспетчер submit(fn) забезпечив асинхронне виконання операцій у фоновій черзі подій. Метод адаптації .get(timeoutMs) дозволив здійснювати безпечне блокуюче очікування з гарантованим захистом від зависань через механізм TimeoutError.

Дворівнева верифікація повністю підтвердила надійність розробленої системи:

- Автоматизований набір із 14 модульних тестів Jest підтвердив коректність роботи переходів станів, обробки помилок, ланцюжків .then() та адаптації вкладених об'єктів.
- Практичні тести через HTTP-сервер та утиліту curl наочно продемонстрували відповідність часових характеристик: паралельне обчислення пулу завдань у CustomFuture.all скоротило загальний час обробки до часу найповільнішої операції (~2 с замість 4.5 с при послідовному виконанні), а комбінатор CustomFuture.race коректно завершився за 1 секунду за результатом найшвидшого воркера.

Код програми

Код до даної лабораторної роботи розміщений у репозиторії https://github.com/MrCatman137/Cloud_Technologies у директорії Lab_8 та має повну інструкцію запуску даної програми.